

1. Flujo de Información y Malware

Esta unidad retoma el problema del flujo de información (ya visto informalmente al hablar de políticas) y lo formaliza con la **entropía**. Cierra el bloque de seguridad en aplicaciones y luego cambia de perspectiva (la del atacante) para introducir el **malware**. Lectura recomendada: Bishop, *Computer Security: Art and Science*, capítulos 16 (parte 1) y 17.

1.1. Flujo de información: el problema

Las políticas de seguridad rara vez se expresan como permisos sobre objetos. Una política típica dice: “*un empleado no puede ver los sueldos de otros empleados*”. El profesor remarca que ese requisito se refiere a **la información en sí**, no al lugar donde la información está en un momento puntual.

Definición: Control de acceso vs. control de flujo

Control de acceso: limita el acceso de un *sujeto* a una *operación* puntual sobre un *objeto* puntual. Como los objetos contienen información, indirectamente limita el acceso a información.

Control de flujo: estudia cómo la información *se mueve* a través del sistema. La información no es estática: **se actualiza y se puede copiar**.

El control de acceso **no impide copiar la información a otro lugar**: un usuario con permiso de lectura sobre su sueldo puede leerlo y escribirlo en un objeto que otros puedan ver (camino de lecturas y escrituras).

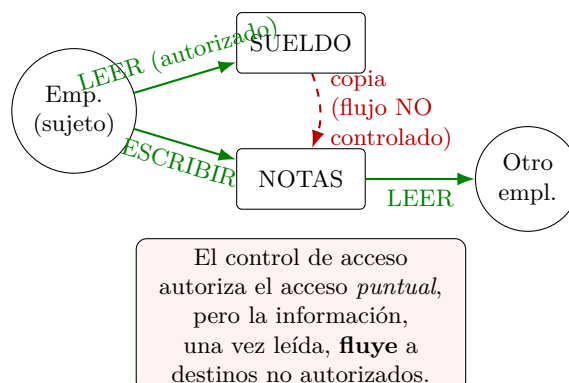
! Importante — remarcado en clase

Las políticas **por lo general restringen el flujo de información y no el acceso a objetos**. Comparar: “evitar que un empleado sepa el sueldo de otro” (flujo) vs. “evitar que un empleado acceda a la base de datos de sueldos” (acceso). El control de acceso *sirve*, pero son **mecanismos abiertos**: aseguran una parte del problema, no todo. Deben ser **complementados**.

► En el parcial

Pregunta clásica de verdadero/falso:

“Una matriz de acceso bien diseñada evita el flujo de información no deseado” → **FALSO**. El control de acceso no controla el flujo. Sí limita el acceso puntual, pero no impide que la información, una vez leída legítimamente, se copie o fluya a otro lugar (directa o indirectamente).



La matriz de acceso autoriza operaciones puntuales (flechas verdes) pero no controla el flujo: el dato leído se copia y se propaga (flecha roja punteada) hacia un destino no autorizado.

Metáfora de capas (cebolla / queso suizo). Como los mecanismos son abiertos, la estrategia es superponer capas de seguridad. Cada capa tiene “agujeros”, pero se apilan tantas que se espera que los agujeros no coincidan en el mismo lugar y unas tapen las de las otras (metáfora del queso suizo).

1.2. Confinamiento y aislación total

Definición: Problema del confinamiento

Dificultad de asegurar que un programa o proceso (*sujeto*) no transmita información a otra entidad (*objeto*) a la que no está autorizado a acceder, ya sea de manera **directa o indirecta**. Es el problema de controlar la información *en tránsito* (cuando se está moviendo). Es un problema complicadísimo de resolver.

Si se pudiera **confinar** el flujo en un ambiente totalmente controlado, no habría riesgos de confidencialidad ni integridad. El modelo **Bell–LaPadula** define un modelo de seguridad que, en un ambiente cerrado donde se aplica al 100 %, da garantías de confidencialidad respecto al flujo.

Definición: Modelo ideal: Aislación total

Requisitos: (1) un proceso no puede comunicarse con otros procesos; (2) un proceso no puede ser observado. **Consecuencia:** el proceso no revela información.

“El sistema más seguro del mundo corre en una computadora que está apagada, metida en una caja fuerte, enterrada en el fondo del océano, sin que nadie sepa dónde está.”

— dicho en clase

El problema es que la aislación total **le quita toda utilidad al sistema** (no se le puede dar entrada ni observar la salida). En la práctica:

- El requisito (1) a veces se logra (red cerrada, sin Internet — “air gap”).
- El requisito (2), **no poder ser observado, es lo más difícil**. Todo proceso usa recursos **observables**: memoria, ciclos de CPU, espacio en disco, ancho de banda. Observándolos se puede extraer información del proceso y de su resultado.

1.3. Canales encubiertos y canales laterales

Definición: Canal encubierto (covert channel)

Método de comunicación usado para transferir información de manera **oculta**, aprovechando vías **no diseñadas** para la transmisión de datos. Como esas vías no estaban pensadas para transmitir, es muy poco probable que tengan los controles que sí tienen los canales normales; por eso sirven para evadir controles de seguridad y transmitir sin detección.

Pueden aparecer **intencionalmente o sin intención**. Tipos:

- **Canales de temporización (Timing Channels):** codifican datos en la variación de tiempo entre eventos. Se muestrea el canal a intervalos regulares y según cómo cambia la lectura se infiere información. Ejemplos: latencia de red, carga de CPU, tiempos de respuesta de aplicaciones. (Caso real mencionado: *traffic shaping* de proveedores de Internet, que analizan la forma de las tramas sin ver el contenido.)

- **Canales de almacenamiento (Storage Channels):** usan áreas no convencionales o no reguladas para guardar datos: espacio de relleno en un archivo, atributos/metadata (fecha de creación, modificación, acceso), bits no usados o reservados en estructuras de datos y protocolos (lugares donde debería ir un valor aleatorio y no lo es tanto).

Ejemplos de canales encubiertos (clase).

- Exfiltración codificando los **sonidos de una impresora** (micrófono de alta fidelidad que reconstruye lo impreso).
- **ICMP & DNS Tunneling:** usar el protocolo DNS para exfiltrar. Como casi todo Internet necesita resolución de nombres, los firewalls dejaban pasar DNS. Idea: quien controla un dominio y su servidor DNS hace que la máquina interna haga búsquedas de subdominios cuyo nombre es (p. ej. en base64) parte de los datos a exfiltrar; esas consultas llegan al servidor del atacante y se rearma la información arbitraria, pasando los controles.
- Emanaciones electromagnéticas; láser sobre una ventana midiendo vibración para reconstruir el sonido de una habitación; *fingerprinting* por forma de tipeo.

Definición: Ataque de canal lateral (Side-Channel Attack)

Explota las **emisiones físicas y temporales** de un sistema para extraer información sensible. **No** se basa en vulnerabilidades clásicas de software ni en errores de autenticación/autorización, sino en la **observación de efectos colaterales del hardware** durante su funcionamiento (tiempos de ejecución, consumo de energía, radiación electromagnética, ruido acústico).

Es especialmente relevante en **criptografía**: toda la seguridad se apoya en que la clave no se filtre, y estos ataques suelen apuntar a **recuperar la clave**. Si se recupera, “se cae toda la seguridad como un castillo de naipes”.

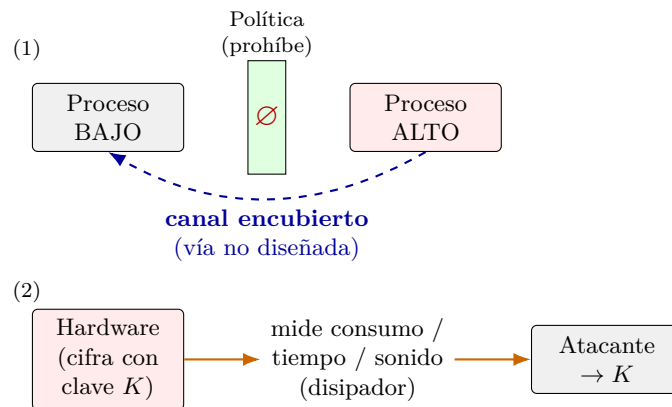
Mecanismo de un ataque por timing sobre cripto. Los algoritmos (y los compiladores) tienen bloques con duración **variable** en ciclos de reloj (p. ej. multiplicar/exponenciar números grandes depende de su tamaño). Midiendo con precisión muchas ejecuciones (sampleo estadístico para sacar el ruido) y modelando el algoritmo, se va infiriendo la clave; combinado con ataque de texto plano conocido, se controla un número y se usa la diferencia de tiempos para despejar la incógnita. No hace falta éxito total: alcanza con **sesgar las probabilidades** de algunos bits y completar el resto por fuerza bruta (p. ej. de 128 bits, sesgar 40 bits deja 2^{88} pruebas en lugar de 2^{128}).

Otros side-channels famosos. Análisis de consumo de energía (rompió históricamente casi todas las *smart cards* con clave criptográfica embebida, como tarjetas de transporte/acceso); Spectre, Meltdown, Rowhammer (hacer vibrar dos filas de memoria para alterar el bit del medio), Rambo (variaciones electromagnéticas convertidas en radiofrecuencia captable desde afuera). Cobraron relevancia con los *hyperscalers* (AWS, Google): en una misma máquina física conviven VMs de distintos clientes, y desde una VM se podría leer información de otra.

△ Cuidado — error típico

No confundir canal encubierto con ataque de canal lateral:

- **Canal encubierto:** es el *medio* (flujo de información no diseñado para transmitir). Se centra en *evadir* controles para transmitir información oculta.
- **Ataque de canal lateral:** es *explotar* un canal encubierto para extraer información sensible (recolectar datos observando el comportamiento físico del hardware). “Dos caras de la misma moneda.”

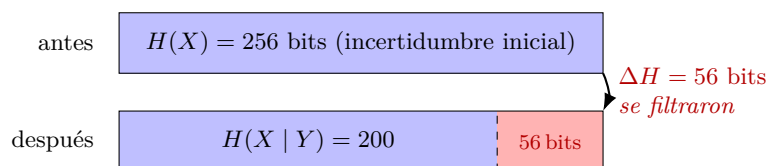


(1) Canal encubierto: dos procesos (alto/bajo) que la política separa, pero que se comunican por una vía no prevista. (2) Ataque de canal lateral: observar el consumo/tiempo/sonido del hardware (caso del disipador) para reducir la entropía de la clave y recuperarla.

► En el parcial

Ejercicio clásico (Recup 2023): si la entropía de una clave de 256 bits se reduce a 200 bits al registrar el sonido del disipador del hardware, ¿hay flujo de información?

SÍ. La reducción de entropía *es* flujo de información. El sonido del disipador es una vía no diseñada para transmitir datos $\Rightarrow$ canal encubierto; explotarlo para sacar info de la clave es un **ataque de canal lateral (side-channel)**. Pasó de $H = 256$ a $H = 200$: se filtraron 56 bits de información sobre la clave. La forma de demostrarlo formalmente es justamente que H (incertidumbre sobre la clave) *disminuyó*.



La reducción de la entropía de la clave de $H(X) = 256$ a $H(X | Y) = 200$ (franja roja = 56 bits) es información que se filtró: hubo flujo de información hacia el atacante.

Mitigación. Resolver todos los canales (incluidos los que no conocemos) es “misión imposible”; sí se puede **mitigar** los conocidos. Las dos familias de control de flujo son: en tiempo de **compilación** y en tiempo de **ejecución** (ver más abajo).

1.4. Entropía como medida de información

Claude Shannon (hace casi 90 años) dio una forma de convertir en **un solo número** la cantidad de información que puede circular por un canal discreto.

Definición: Entropía $H(X)$

Sea X una variable aleatoria discreta (V.A.D.) que toma valores $x_1, \dots, x_n$:

$$H(X) = - \sum_{i=1}^n p(x_i) \log p(x_i)$$

(con log en base 2, da bits). Mide la **incertidumbre** al determinar el valor de X : cuánta información necesitaríamos conocer para explicar completamente el contenido del canal (medida de *falta* de información / incertidumbre).

Extremos:

- **Mínimo** $H(X) = 0$: cuando algún $p(x_i) = 1$ (y el resto 0). Sabemos *exactamente* qué se transmite; no hay información extra. (Cada término vale 0: o porque $p = 0$, o porque $p = 1 \Rightarrow \log 1 = 0$.)
- **Máximo**: cuando $p(x_i) = 1/n$ para todo i (**distribución uniforme**). Es lo más incierto. Para n valores equiprobables, $H(X) = \log n$.

La entropía es **continua**: cuantifica también los escenarios intermedios (conocemos algo, pero no todo).

Definición: Entropía condicional $H(X | Y)$

Sea X con valores $x_1, \dots, x_n$ e Y con valores $y_1, \dots, y_m$:

$$H(X | Y = y) = - \sum_{i=1}^n p(x_i | y) \log p(x_i | y) \quad H(X | Y) = \sum_{j=1}^m p(y_j) H(X | Y = y_j)$$

Es el promedio ponderado de las entropías condicionales. Mide cuánta incertidumbre queda sobre X sabiendo Y .

1.5. Flujo de información vía entropía

Definición: Flujo de información (formal)

Sea s el estado inicial del sistema y t el estado tras ejecutar comandos $c_1, \dots, c_n$. Sean x_s, y_s valores de objetos en el estado s y y_t el valor de y en el estado t . **Hay flujo de información de x a y si:**

$$\begin{aligned} H(x_s | y_t) &< H(x_s | y_s), & \text{si } y \text{ existe en el estado } s \\ H(x_s | y_t) &< H(x_s), & \text{si } y \text{ NO existe en el estado } s \end{aligned}$$

Idea: cuantifico lo que me falta conocer de x al principio y al final (observando y). Si al final **conozco más** de x (la entropía **bajó**), hubo **transferencia de información**. Si no debiera haber flujo y la medición cambia, hay un problema.

△ Cuidado — error típico

Cuándo usar cuál fórmula (esto se equivoca seguido):

- Si la variable destino y **ya existía** al principio, se compara contra la **condicional** $H(x_s | y_s)$.
- Si y **se crea** durante la ejecución, se compara contra la entropía **absoluta (no condicional)** $H(x_s)$ del estado inicial.

Sin flujo, debería dar **mayor o igual**. El “=” es claro (si y no tiene nada que ver, H no cambia). Puede incluso *aumentar* la entropía si una sentencia “pisa” (sobrescribe) el valor de la variable, destruyendo información.

1.5.1. Flujo directo

Se sigue el flujo **sentencia por sentencia**, midiendo cómo cambia la información de cada variable. Ejemplo de la teoría: comando $y = x + z$, con

$$0 \leq x \leq 7 \text{ (8 valores equiprobables)}, \quad Z = \{p(z=1) = 0,5, p(z=2) = 0,25, p(z=3) = 0,25\}$$

Antes del comando (8 valores equiprobables, $p = 1/8$):

$$H(x) = -\sum_{i=1}^8 \frac{1}{8} \log \frac{1}{8} = -8 \cdot \frac{1}{8} \log \frac{1}{8} = \log 8 = 3$$

Conociendo y luego del comando, x solo puede valer $\{y-1, y-2, y-3\}$ con las probabilidades de z :

$$H(x | y) = -\frac{1}{2} \log \frac{1}{2} - 2 \cdot \frac{1}{4} \log \frac{1}{4} = \frac{1}{2} + 2 \cdot \frac{1}{2} = 1,5$$

Como $H(x) = 3$ y luego $H(x | y) = 1,5 < 3$: **hubo traspaso de información de x a y .**

1.5.2. Flujo indirecto

La información puede fluir aunque x e y **no aparezcan en la misma asignación**, por la estructura de control.

Por asignación condicional. `if (x == 0) y = 1; else y = 0;` Con $x \in \{0, 1\}$, $p(x=0) = 0,5$: $H(x) = 1$ y $H(x | y) = 0$ (conociendo y se deduce x). **Hay traspaso de información.** (Canal “espacial” por comportamiento.)

Por comportamiento / terminación. `while (x == 0) {}` No hay ninguna asignación. Con $x \in \{0, 1\}$, $p(x=0) = 0,5$, se define una variable observable $y = 0$ si el programa termina (y otro valor si no). $H(x) = 1$, $H(x | y) = 0$: **hay traspaso de información** observando si el programa termina o no (canal temporal por comportamiento).

! Importante — remarcado en clase

El flujo de información **siempre existe** en un programa — es necesario para que funcione. Por eso, el seguimiento de flujo no busca verificar *todo* flujo en toda configuración, sino **casos críticos puntuales** (p. ej. que una variable que contiene una **clave criptográfica** no pueda inferirse midiendo otras variables observables).

1.5.3. Límites (compartidos con la verificación formal)

- **No es computable** verificar todo programa (límite teórico; los no computables suelen ser programas que nunca terminan).
- Con **interactividad** (inputs externos: teclado, mouse) la cantidad de casos **explota exponencialmente**. Se limita a sistemas chicos y cerrados.
- Los **ciclos** son un problema. Técnica: *loop unrolling* (desenrollar el ciclo un número fijo de veces, p. ej. hasta 10; suele alcanzar porque los problemas estructurales aparecen en pocos ciclos), pero queda el cabo suelto de iteraciones más profundas.

1.6. Control de flujo: compilación vs. ejecución

- **Tiempo de compilación (compile-time enforcement):** analiza la propagación de información directamente en el código fuente (las técnicas de entropía de arriba; relacionado con verificación formal). Costoso y complejo → limitado a componentes muy sensibles. (Ej. Tanenbaum armó un microkernel con demostración formal de seguridad.)
- **Tiempo de ejecución (run-time enforcement):** control dinámico (**sandboxing**) y **aislación** (virtualization). Son los **métodos más utilizados**.

1.6.1. Aislamiento (en ejecución)

Virtualización (Virtual Machines). Se simula a la aplicación un **entorno de ejecución sintético**, separado del real y aislado por definición. Las VMs usan **hipervisores** para emular hardware:

- **Bare-metal:** KVM, VMware ESXi, Hyper-V.
- **Hosted:** VirtualBox, VMware Workstation.

Cada VM tiene su **propio kernel** $\Rightarrow$ aislamiento fuerte (no comparten CPU). Pero es **pesadísimo**: emula todo el ambiente.

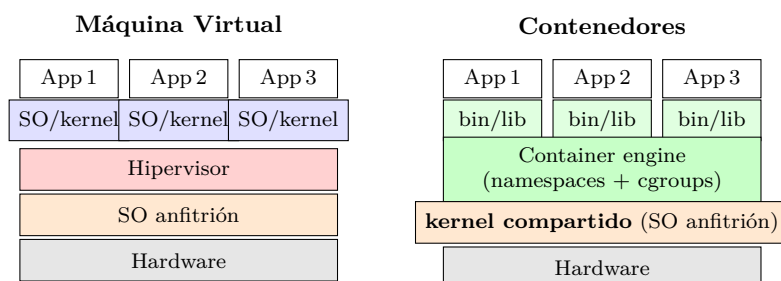
Sandboxing. Más liviano: en vez de simular todo, **arbitra las interacciones** (entrada/salida) del proceso con su entorno y las modifica para aislarlo, decidiendo dinámicamente si lo deja seguir.

- **A nivel SO:** el sistema operativo deriva las interacciones a un mecanismo de control. **Linux:** **namespaces** (aíslan procesos, usuarios, red, filesystem) y **cgroups** (controlan/limitan CPU, memoria, red). **Windows:** *User Mode Scheduling* (similar a namespaces) y *Job Objects* (agrupan procesos como una unidad para límites y prioridades).
- **Reescritura del programa:** agregar los controles antes de cada interacción.
- Ejemplo simple: **chroot** en Unix — lanza un proceso con una vista “encajonada” del filesystem (un directorio funciona como raíz). Usado también por los instaladores de paquetes Unix: corren scripts de terceros en un sandbox y luego copian solo los archivos que no resultan sospechosos.

Contenedores. Aprovechan **namespaces + cgroups** (**Docker** configura ambos) para aislar aplicaciones: **comparten el mismo kernel del SO anfitrión** pero mantienen separación lógica de red, procesos y filesystem. Eliminan la necesidad de un SO completo por instancia (más livianos que las VMs). Windows soporta contenedores nativos usando características de Windows o Hyper-V.

△ Cuidado — error típico

Diferencia clave VM vs. contenedor (suele preguntarse): la **VM** replica todo el stack (incluye un **SO/kernel propio** por instancia) $\Rightarrow$ aislamiento fuerte pero pesado. El **contenedor comparte el kernel** del host $\Rightarrow$ liviano, pero comparte más recursos (quedan canales no aislados: memoria, CPU compartidas).



VM: cada instancia trae su propio SO/kernel sobre el hipervisor (aislamiento fuerte, pesado). Contenedor: las apps comparten el mismo kernel del anfitrión y se separan con namespaces/cgroups (liviano, pero comparten más recursos).

1.7. Malware

Definición: Malware (Malicious Software)

Cualquier programa **diseñado** para causar daño, atravesar un control o extraer información. Lo que los distingue es haber sido concebidos **exclusivamente con un fin malicioso**. Suelen aprovechar vulnerabilidades del software. Un ataque usualmente usa **múltiples tipos** de malware para cada parte del ciclo del ataque.

1.7.1. Tipos de malware

Virus / Gusano (Worm): su rasgo característico es la capacidad de **propagarse** (auto-replicarse).

El virus infecta una máquina/proceso y usa los medios disponibles para propagarse a otras: explotando una vulnerabilidad remota o reescribiendo un ejecutable/recurso compartido. Un virus suele tener dos componentes: **replicación** + **ejecución** (*payload*); según su payload recibe otro nombre. El **worm** se especializa en propagarse (su “única misión” es multiplicarse y ocupar recursos), inclusive por email; asociado a **denegación de servicio**.

- Primer gusano: nació por **error** (Berkeley), se multiplicaba en cada máquina hasta saturar la memoria y tirar el SO.
- **Melissa (1999):** se propagaba por email.
- **SQL Slammer:** infectó 75,000 computadoras en 10 minutos explotando bases SQL de Microsoft expuestas a Internet.

Troyano: malware **oculto dentro de otro software** que parece benigno y de interés para la víctima (caballo de Troya). Sirve para establecer al atacante dentro del equipo. Ejemplo: **PC-Write (1986)**, shareware de editor de texto que borraba todos los archivos. Frecuente en cracks de Windows/Office y en videojuegos móviles modificados.

Rootkit: software diseñado para **ocultar** malware dentro de la máquina infectada. Usa ofuscación, encriptación, reescritura y reemplazo de herramientas del sistema; toma **contramedidas activas** (p. ej. interceptar *system calls* para no aparecer en **ps** ni al listar directorios). Opera a nivel **kernel o usuario**; los más avanzados se meten en la **BIOS/bootloader** y persisten aunque se formatee el disco (por eso existe Secure Boot / BIOS firmada). Muy difíciles de detectar por antivirus.

- **NTRootkit (1999):** primer rootkit para Windows NT.
- **Stuxnet (2010):** se escondía y propagaba por meses; solo actuaba al llegar a las máquinas (con firma conocida) de una planta nuclear de Irán (aisladas de Internet), sabotando máquinas industriales.

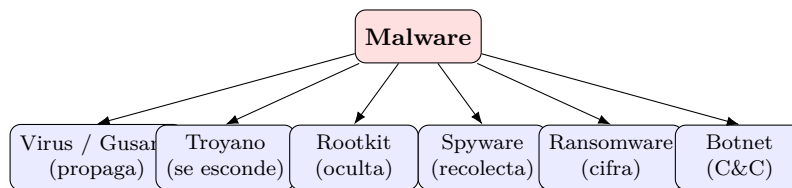
Spyware: software diseñado para **recolectar información** del usuario infectado. Se especializa en ocultar la información dentro del activo y suele establecer alguna forma de **covert channel (túnel)**. Algunas empresas lo desarrollan a pedido de gobiernos para vigilancia/espionaje (ej. mencionado: Palantir).

Ransomware: bloquea el acceso del usuario a su información (típicamente la **cifra**) y pide **dinero (rescate)** para restaurarla; otros exfiltran y piden rescate para no publicar. Apareció como uso “creativo” de las criptomonedas (cobro descentralizado y difícil de rastrear).

- **CryptoLocker (2013):** por mail; ~USD 3M robados ese año.
- **WannaCry (2017):** explotó una vulnerabilidad conocida de Windows, se distribuyó por la red local; > 200K máquinas, costo estimado USD 4B.

Botnet: despliega **bots/agentes multiuso** (máquinas “zombi”) que quedan a la escucha de instrucciones de un **Command & Control (C&C)**. Las máquinas infectadas se usan para

ataques distribuidos (DDoS), minería de cripto, accesos anónimos, distribuir spam, robar credenciales. La infraestructura C&C es un sistema **distribuido, descentralizado, sin único punto de falla**. **Srizbi (2008)**: la botnet más grande documentada, responsable de ~la mitad del spam mundial (60B mails/día).



Taxonomía de los tipos de malware vistos en clase. Virus/gusano y troyano describen cómo llega/se propaga; el resto describe el payload o la función.

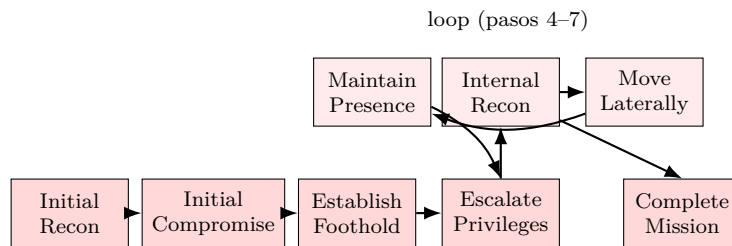
► En el parcial

Definiciones que se preguntan (verdadero/falso o emparejar):

- Virus/Worm → se *propaga*. Troyano → se *esconde* dentro de software benigno. Ni virus ni troyano dicen qué hace una vez adentro: eso es el **payload** (ransomware, C&C, spyware...).
- Rootkit → se *oculta* con contramedidas activas (no es un mecanismo de propagación, sino de ocultamiento/persistencia). Puede ocultar tanto virus como troyanos u otros programas. Se combinan: un virus se propaga, en cada máquina se enquista como rootkit y ejecuta un ransomware.

1.7.2. Ciclo de ataque del malware (Malware attack cycle)

1. **Reconocimiento inicial (Initial Recon)**: se identifica a la víctima y se recopila información (puntos débiles, responsables clave, redes sociales de empleados; se establecen relaciones falsas).
2. **Compromiso inicial (Initial Compromise)**: el malware se “*weaponiza*” (se empaqueta para ser ejecutado por la víctima) y se compromete una o más máquinas con un ataque dirigido. Puede durar varios días.
3. **Establecerse (Establish Foothold)**: establece comunicación con su C&C, asegura persistencia y almacenamiento; baja y compone nuevas herramientas. Suele durar minutos tras la infección inicial.
4. **Escalar privilegios (Escalate Privileges)**: eleva los privilegios obtenidos usando vulnerabilidades locales o remotas. Minutos.
5. **Reconocimiento interno (Internal Recon)**: mapea otras máquinas y usuarios de interés; prueba comunicación/visibilidad; baja nuevos malwares.
6. **Movimiento lateral (Move Laterally)**: explotación remota de vulnerabilidades para incrementar la base infectada.
7. **Mantener la presencia (Maintain Presence)**: persiste en las nuevas máquinas. (Pasos 4–7 forman un **loop**.)
8. **Completar la misión (Complete Mission)**: ejecuta su payload. Va exfiltrando o acumulando la información hasta enviarla en horarios de menor monitoreo; en ransomware, infecta la mayor cantidad posible de máquinas antes de activar el bloqueo.



Ciclo de ataque del malware: las primeras fases son secuenciales; escalada de privilegios, reconocimiento interno, movimiento lateral y mantener presencia forman un loop que expande la base infectada antes de completar la misión (payload).

Lo interesante: desde el reconocimiento inicial hasta el loop de escalamiento, el ciclo es **genérico** (independiente del payload final). Por eso existen **kits de desarrollo de malware** vendidos como producto en el mercado gris (aunque a veces son ellos mismos troyanos, y al ser genéricos tienen más chance de ser detectados).

△ Cuidado — error típico

Errores que penalizan en esta unidad:

- No reconocer que una **reducción de entropía es flujo de información** (caso del disipador / side-channel).
- Afirmar que el control de acceso (matriz de acceso) evita el flujo de información: **no lo evita**.
- Usar la fórmula de flujo equivocada: condicional $H(x_s | y_s)$ si y ya existía; absoluta $H(x_s)$ si y se crea durante la ejecución.